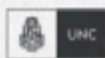


CERTIFICACIÓN UNIVERSITARIA



FCEFYN



Manual para el
alumno

Calidad



WE WANT
SKILLS!

mundosE

Manual para el alumno: Calidad.....	2
Objetivo del encuentro:.....	2
1. Calidad a lo largo del ciclo de vida.....	3
2. Pruebas automatizadas e integración en CI/CD.....	4
3. Control de calidad del código: estático, reviews y code smells.....	5
4. Seguridad y vulnerabilidades en el pipeline.....	5
5. Pruebas de rendimiento y carga.....	6
6. Métricas de calidad y mejora continua.....	7
Bibliografía.....	8

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este encontrarás un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Calidad

Objetivo del encuentro:

Comprender la importancia de la calidad a lo largo del ciclo de vida del *software* e integrar prácticas y herramientas (pruebas automatizadas, análisis estático, seguridad y rendimiento) en pipelines de CI/CD para entregar productos robustos y seguros.

Contenido a desarrollar:

- ✓ Panorama general de calidad en el SDLC: distinción QA vs. QC, concepto de "calidad como código" y contrato de integración con CI/CD.
- ✓ Pruebas automatizadas: unitarias, de integración, funcionales/E2E y de aceptación; cobertura (JaCoCo/Istanbul/pytest-cov), nociones de mutación; ejecución e informes en pipelines (reportes JUnit/HTML, umbrales y fallos).
- ✓ Control de calidad del código: análisis estático con ESLint, Checkstyle y SonarQube; *code smells* frecuentes y refactor; *quality gates* y políticas de merge; revisión de código con PR y checklists.
- ✓ Seguridad y vulnerabilidades en el pipeline: SAST (CodeQL/Semgrep), DAST (OWASP ZAP) y SCA (Dependency-Check/Trivy); SBOM (CycloneDX/SPDX), firmas con Sigstore/Cosign y nociones de SLSA; criterios de severidad y *waivers*.
- ✓ Métricas de calidad y mejora continua: cobertura, deuda técnica, duplicación, complejidad, *defect density*; indicadores de proceso (lead time, tiempo de ciclo, MTTR); tableros y fijación de objetivos incrementales.

1. Calidad a lo largo del ciclo de vida

En este encuentro entendemos la calidad como una propiedad transversal del desarrollo: no se agrega al final, sino que se construye desde el primer commit. Para lograrlo, combinamos dos enfoques complementarios. Por un lado, **QA (Aseguramiento de la Calidad)**: definir estándares, criterios de aceptación, políticas de branch y revisiones; por otro, **QC (Control de Calidad)**: ejecutar pruebas, análisis y mediciones que verifiquen que el producto cumple lo esperado. En la práctica, esto significa que cada cambio que proponemos en una *pull request* (PR) pasa por un conjunto de chequeos automáticos antes de poder integrarse.

Las **pruebas automatizadas** son la primera línea de defensa. Las **unitarias** verifican funciones o clases pequeñas; las de **integración** validan la interacción entre módulos; las **funcionales o end-to-end (E2E)** ejercitan la aplicación como un usuario real; y las de **aceptación** certifican que se satisfacen criterios acordados con el cliente o con la cátedra. En CI/CD, estas pruebas se ejecutan en cada PR y en ramas principales, generando reportes que quedan como evidencia.

El **análisis estático** es otra pieza central. Herramientas como SonarQube, ESLint o Checkstyle examinan el código sin ejecutarlo para detectar **bugs**, **vulnerabilidades**, duplicación y **code smells** (indicadores de diseño pobre que complican el mantenimiento). A diferencia de las pruebas, que confirman comportamientos, el análisis estático hace cumplir estándares de calidad y estilo, y calcula métricas útiles como **deuda técnica** y **cobertura**. Con **quality gates**, el equipo define condiciones mínimas —por ejemplo “cobertura en código nuevo $\geq 80\%$ y sin vulnerabilidades críticas”— que determinan si una PR puede integrarse.

La **seguridad** atraviesa todo: analizamos dependencias (*Software Composition Analysis, SCA*) para encontrar CVEs conocidas; aplicamos **SAST** (análisis del código fuente) y **DAST** (escaneo dinámico de la aplicación en ejecución); generamos **SBOM** (inventario de componentes) y firmamos artefactos para garantizar origen e integridad. Finalmente, consideramos los **requisitos no funcionales** de **rendimiento y carga**, con herramientas que simulan usuarios concurrentes para medir latencia, *throughput* y estabilidad.

Para que todo esto sea sostenible, usamos el principio de **“calidad como código”**: las reglas de linters, los umbrales de los *gates*, las tareas de *test* y los manifiestos de seguridad viven en el repositorio junto con el programa. Así es posible auditar cambios, revertir configuraciones y replicar resultados. El pipeline de CI/CD se convierte en un **contrato**: si se cumplen los chequeos, se avanza; si no, se detiene, se corrige y se vuelve a intentar. Este ciclo rápido de feedback permite aprender sin acumular trabajo pendiente.

Recursos para profundizar:

- SonarSource © (2025) SonarQube Docs: <https://docs.sonarsource.com/>
- ESLint Docs: <https://eslint.org/docs/latest/>
- Checkstyle: <https://checkstyle.sourceforge.io/>
- OWASP Top Ten: <https://owasp.org/www-project-top-ten/>

2. Pruebas automatizadas e integración en CI/CD

Las pruebas automatizadas brindan confianza y evitan regresiones. Empezamos por **unitarias**, donde el objetivo es aislar una unidad (función/método) y comprobar entradas y salidas con aserciones. En Java, **JUnit** es estándar; en Python, **pytest** destaca por su sistema de *fixtures*; en JavaScript/TypeScript, **Jest** y **Mocha** son comunes. Las **pruebas de integración** verifican que componentes colaboran correctamente (por ejemplo, un servicio que invoca un repositorio y una API externa). Requieren preparar datos, *mocks* y *stubs* para aislar dependencias costosas. Las **funcionales/E2E** se enfocan en flujos completos desde la interfaz o la API, y suelen ser menos numerosas debido a su costo.

En pipelines, las *suites* se ejecutan **en paralelo** para acelerar el *feedback*; el resultado es un reporte con casos pasados/fallidos y, en muchos lenguajes, un archivo XML/JUnit que las plataformas de CI pueden interpretar y mostrar. Para medir **cobertura**, integramos herramientas específicas: **JaCoCo** (Java), **Istanbul/nyc** (JS/TS) y **pytest-cov** (Python). La cobertura es un indicador, no un fin: 100 % no garantiza ausencia de fallas; se busca un equilibrio que cubra el código crítico y los caminos de error. Una práctica útil es combinar cobertura con **tests de mutación** que modifican el código para evaluar si los *tests* detectan cambios (más exigente, pero muy formativo).

En la configuración mínima de CI/CD, definimos estos pasos: (1) **instalación** de dependencias; (2) **lint**; (3) **build**; (4) **unit/integration tests** con generación de reportes; (5) **coverage**; (6) publicación de artefactos de reporte; (7) notificación en PR (estado verde/rojo). Si incorporamos SonarQube, enviamos los resultados de pruebas y cobertura para enriquecer las métricas y aplicar el **quality gate**.

Recursos para profundizar:

- pytest Docs: <https://docs.pytest.org/en/stable/>
- JUnit 5 User Guide: <https://docs.junit.org/current/user-guide/>
- Jest Docs: <https://jestjs.io/docs/getting-started>

- Mocha: <https://mochajs.org/>
- JaCoCo: <https://www.jacoco.org/jacoco/trunk/doc/>
- Istanbul/hyc: <https://istanbul.js.org/>

3. Control de calidad del código: estático, reviews y code smells

El análisis estático ayuda a **prevenir** errores y a mantener un estilo consistente. **ESLint** aplica reglas sobre JavaScript/TypeScript (y se complementa con **typescript-eslint** para reglas basadas en tipos). **Checkstyle** estandariza formato y convenciones en Java. **SonarQube** agrega una visión integral: clasifica hallazgos en **bugs**, **vulnerabilidades** y **code smells**; mide **duplicación**, **complejidad**, **deuda técnica** y **cobertura**; y permite definir **quality gates** como política. En este encuentro, nos enfocamos en entender **qué es un code smell** (p. ej., función demasiado larga, clases “Dios”, parámetros excesivos) y cómo **refactorizar** para mitigarlos.

La revisión de código es el complemento humano del análisis estático. Una buena **PR** explica el problema, el enfoque, y limita el tamaño para facilitar comentarios. El revisor evalúa diseño, legibilidad, pruebas y seguridad (¿estamos validando entradas?, ¿se gestionan secretos adecuadamente?). Establecer **listas de chequeo** (*checklists*) mejora la consistencia entre equipos.

Los **quality gates** transforman la cultura del equipo: el merge no depende de interpretaciones *ad-hoc*, sino de criterios explícitos. Una política inicial puede ser: en **código nuevo**, *No new critical issues*, **coverage $\geq 80\%$** , **duplicación $\leq 3\%$** , **sin vulnerabilidades críticas**. Si el proyecto ya tiene deuda, se centra el gate en **nuevo código** para permitir avanzar sin heredar problemas históricos.

4. Seguridad y vulnerabilidades en el pipeline

La seguridad no se “agrega” al final: se incorpora en cada etapa del pipeline. El primer paso es conocer nuestras **dependencias** y sus riesgos: con **OWASP Dependency-Check** o **Trivy** generamos reportes de CVEs y fallamos el build si aparece una severidad crítica. En paralelo, aplicamos **SAST** con **CodeQL** o

Semgrep para detectar patrones inseguros en el código (inyección, deserialización peligrosa, uso inseguro de APIs). Tras construir la aplicación, podemos ejecutar **DAST** con **OWASP ZAP** contra un entorno temporal para detectar problemas de autenticación, XSS, CSRF y más.

Para dar trazabilidad, generamos un **SBOM** (CycloneDX o SPDX) en el *build* y lo adjuntamos al release. Con **Cosign** firmamos imágenes y binarios para verificar origen e integridad; el entorno de despliegue debe **verificar** esa firma antes de aceptar el artefacto. Como objetivo de mediano plazo, adoptamos prácticas **SLSA** para elevar el nivel de garantía del proceso de construcción (*build* hermético, materiales versionados, provenance verificable).

Recursos para profundizar:

- OWASP Dependency-Check: <https://owasp.org/www-project-dependency-check/>
- Trivy: <https://trivy.dev/>
- OWASP ZAP: <https://www.zaproxy.org/docs/>
- CodeQL: <https://docs.github.com/en/code-security/code-scanning/>
- Semgrep: <https://semgrep.dev/docs/>
- CycloneDX: <https://cyclonedx.org/>
- SPDX: <https://spdx.dev/>
- Sigstore / Cosign: <https://docs.sigstore.dev/about/overview/>
- SLSA: <https://slsa.dev/>

5. Pruebas de rendimiento y carga

Las pruebas de rendimiento miden **tiempos de respuesta**, **consumo de recursos** y **estabilidad** bajo carga. Diferenciamos **pruebas de carga** (comportamiento esperado), **estrés** (hasta romper), **picos** (ráfagas), y **endurance/soak** (larga duración). Elegimos herramientas según el *stack* y la ergonomía: **JMeter** ofrece GUI y gran compatibilidad; **k6** usa scripts JavaScript; **Gatling** aporta un DSL potente en Scala; **Locust** permite describir usuarios en Python; **Artillery** ofrece YAML/Node y buena integración con Playwright para escenarios E2E.

El diseño de escenarios requiere definir **objetivos** (p. ej., mantener P95 < 300 ms a 200 rps), **fases** (ramp-up, steady, ramp-down), **datos** y **métricas** (percentiles de latencia, *throughput*, *error rate*). Se ejecutan idealmente en entornos aislados y reproducibles; los resultados se comparan entre versiones para detectar regresiones.

Recursos para profundizar:

- JMeter: <https://jmeter.apache.org/>
- k6: <https://k6.io/docs/>
- Gatling: <https://gatling.io/docs/>
- Locust: <https://docs.locust.io/>
- Artillery: <https://www.artillery.io/docs>

6. Métricas de calidad y mejora continua

Medir es clave para mejorar. Entre las métricas más útiles están **cobertura** (porcentaje del código ejercitado por pruebas), **deuda técnica** (esfuerzo estimado para remediar *code smells*), **duplicación**, **complejidad**, y **defect density**. También conviene monitorear indicadores de proceso: tiempo de ciclo, tasa de fallas del build, *lead time* para cambios y **MTTR** (tiempo medio de recuperación).

Los **quality gates** permiten traducir estas métricas en decisiones automáticas: si el código nuevo no alcanza el umbral acordado o introduce *issues* críticos, el merge se bloquea. Es esencial **contextualizar**: una cobertura “baja” puede ser aceptable si se compensa con tests de mutación en módulos críticos; una deuda “alta” puede priorizarse por impacto en mantenibilidad. La clave es **iterar**: definir metas realistas por *sprint*, revisar el tablero de calidad y ajustar el proceso.

Actividad propuesta: diseñar, en equipo, un tablero mínimo de calidad con 5 métricas que se puedan obtener automáticamente del pipeline o de SonarQube, fijar un valor objetivo para cada una y planificar acciones concretas para acercarse a esos objetivos en dos iteraciones.

Recursos para profundizar:

- SonarQube — Test Coverage: <https://docs.sonarsource.com/sonarqube-server/10.8/analyzing-source-code/test-coverage/overview/>
- SonarQube — Quality Gates: <https://docs.sonarsource.com/>
- Concepto de deuda técnica (SonarSource): <https://www.sonarsource.com/learn/technical-debt/>

Bibliografía

- Aqua Security. (s. f.). *Trivy Documentation*. <https://trivy.dev/>
- Artillery. (s. f.). *Docs*. <https://www.artillery.io/docs>
- Checkstyle. (s. f.). *Checkstyle*. <https://checkstyle.sourceforge.io/>
- ESLint. (s. f.). *Documentation*. <https://eslint.org/docs/latest/>
- Gatling Corp. (s. f.). *Gatling Docs*. <https://gatling.io/docs/>
- GitHub. (s. f.). *CodeQL Docs*.
<https://docs.github.com/en/code-security/code-scanning/>
- Grafana Labs. (s. f.). *k6 Docs*. <https://k6.io/docs/>
- JMeter. (s. f.). *Apache JMeter*. <https://jmeter.apache.org/>
- Linux Foundation. (2024). *SPDX Specification*. <https://spdx.dev/>
- Locust. (s. f.). *Docs*. <https://docs.locust.io/>
- OWASP. (s. f.). *Dependency-Check*.
<https://owasp.org/www-project-dependency-check/>
- OWASP. (2021). *Top Ten*. <https://owasp.org/www-project-top-ten/>
- OWASP & Ecma. (2024). *CycloneDX Specification*. <https://cyclonedx.org/>
- Semgrep. (s. f.). *Docs*. <https://semgrep.dev/docs/>
- Sigstore. (s. f.). *Cosign Overview*. <https://docs.sigstore.dev/about/overview/>
- SonarSource. (s. f.). *SonarQube Documentation*.
<https://docs.sonarsource.com/>